ONNX Runtime User Guide

Version: 0.1

Release Date: 2026-11-06

Copyright

© 2026 Amlogic (Shanghai) Co., Ltd. (on behalf of itself and its affiliates, hereinafter referred to as "Amlogic"). All rights reserved. No part of this document may be reproduced, distributed, transcribed, or translated into any language, in any form or by any means, without the prior written permission of Amlogic.

Trademarks

Amlogic, and other Amlogic icons are trademarks of Amlogic. All other trademarks and registered trademarks are the property of their respective owners.

Disclaimer

This document may be updated or revised from time to time without notice due to product upgrades or other reasons.

This product is not intended for medical, life-saving, or life-sustaining applications, or for applications in which failure or malfunction of an Amlogic product could reasonably be expected to result in personal injury, death, or severe property or environmental damage.

Circuit diagrams and other information related to Amlogic products in this document are provided only as typical application examples. Complete information sufficient for production design may not be provided. Amlogic has made reasonable efforts to ensure that this document is accurate and timely, but makes no warranty regarding the accuracy or completeness of the information contained herein.

Amlogic products may contain unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic assumes no liability for any vulnerability.

Use of the relevant materials may require a license to intellectual property owned by a third party or Amlogic. This document grants no express or implied representation or warranty regarding intellectual property owned by Amlogic or any other party. Links to third-party websites are provided solely for convenience. Amlogic does not endorse and assumes no responsibility for such websites or any of their practices, including privacy practices, availability, and content.

Under no circumstances shall Amlogic be liable for any claim arising from or related to this document or the use of, or inability to use, this document.

Contact Information

- Website: www.amlogic.cn
- Pre-sales inquiries: contact@amlogic.com
- Technical support: support@amlogic.com

Revision History

Version	Description
0.1 (2026-11-06)	Initial release.

Contents

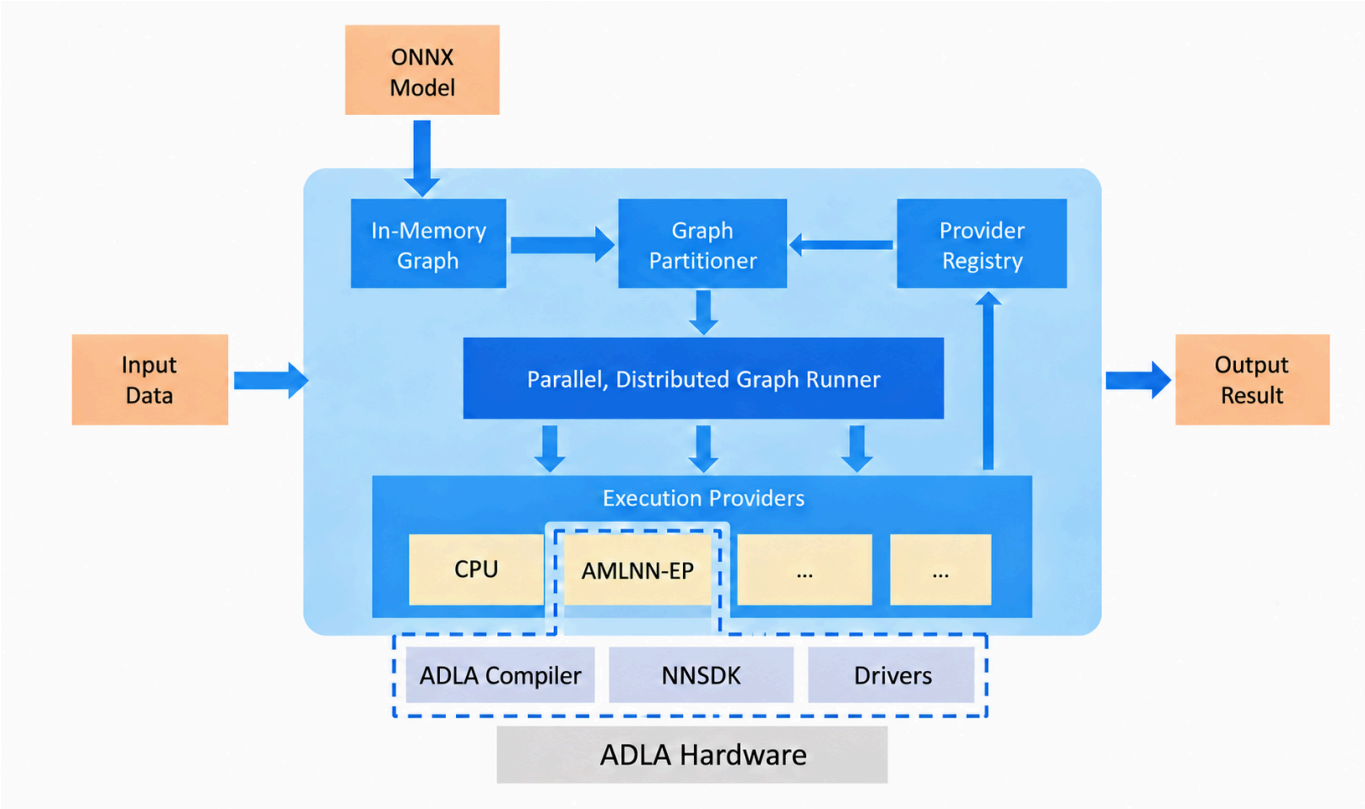
1 Introduction to ONNX Runtime
1.1 ONNX Runtime workflow
1.2 Platform support
2 Development environment verification
2.1 Version verification
2.1.1 Driver version
2.1.2 Runtime version
2.2 System environment
2.3 Platform identification
2.4 Quick validation
3 ONNX Runtime C/C++ deployment
3.1 Install build toolchains
3.1.1 Install CMake and host tools
3.1.2 Install cross-compilation toolchains
3.1.2.1 Android: install and configure the NDK
3.1.2.2 Buildroot: install a GCC cross-toolchain
3.1.2.3 Armbian/Yocto: install a Poky SDK
3.2 Build and run ONNX Runtime
3.2.1 Source download
3.2.2 Build and run
3.2.2.1 Build and run on Android
3.2.2.2 Build and run on Buildroot
3.2.2.3 Build and run on Armbian/Yocto
4 Supported models
5 FAQ

1 Introduction to ONNX Runtime

This guide describes ONNX Runtime deployment on the Amlogic NPU platform. It helps developers quickly complete environment setup, model conversion, inference, and performance validation. The standard ONNX Runtime architecture is extended with the `amlnn` hardware-acceleration Execution Provider (EP), allowing ONNX models to run directly on the specified chip platform, fully utilize NPU compute resources, and lower the barrier to deploying models on edge devices. The guide provides installation steps, API examples, and troubleshooting information to help developers deploy AI applications efficiently.

ONNX Runtime is an open-source, cross-platform, high-performance machine-learning inference and training accelerator led by Microsoft. It is designed to execute models that conform to the ONNX standard and is widely used for production AI model deployment. Project: https://github.com/microsoft/onnxruntime

1.1 ONNX Runtime workflow



The processing flow is as follows:

Step 1: The application submits a floating-point ONNX model and input tensors to ONNX Runtime.
Step 2: ONNX Runtime parses the model and assigns supported graph partitions to the AMLNN EP.
Step 3: The AMLNN EP invokes Core/Compiler to convert the graph to ADLA online on the target device.
Step 4: NNSDK loads the generated ADLA and invokes the NPU to perform inference.
Step 5: The inference results are returned through the AMLNN EP and ONNX Runtime to the application.

Model conversion and inference both run on the target device, so the generated ADLA file does not need to be transferred manually.

1.2 Platform support

This document supports the following product:

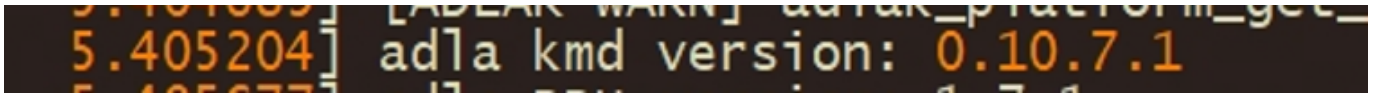
- A311Y3

2 Development environment verification

2.1 Version verification

2.1.1 Driver version

Verify the driver version: Restart the board. From the serial console, or from a CMD window after connecting with `adb shell`, run `dmesg | grep adla`. The output is similar to the following:



The NPU Driver version shown in the example is 0.10.7 (use the first three version components).

The content of this document supports NPU Driver version >= 2.0.0.

2.1.2 Runtime version

This release uses the following runtime components. The versions in the table correspond to the Core API, ADLA Compiler, and NNSDK2 runtime respectively and are not interchangeable.

Component	Version	Verification
`aml_compiler_core`	1.0.2	Use the vendor release information. Applications can call the public `AmlCompilerCoreGetVersion()` API to query the version of the library actually loaded.
ADLA Compiler (`libcompiler_adla.so`)	3.5.3	The online-conversion log must show `Compiler Config: Compiler Version: 3.5.3`.
NNSDK2	v1.1.0	Run the command below for the current target system to read the version from the device's `libnnsdk.so`. The NNSDK2 version in the output must be `v1.1.0`.

The installation path of `libnnsdk.so` differs by system. Run the corresponding command on the target device:

```
# Android
strings /vendor/lib64/libnnsdk.so | grep NNSDK

# Armbian/Yocto or Buildroot
strings /usr/lib/libnnsdk.so | grep NNSDK
```

If the command prints no version information, first verify that `libnnsdk.so` exists at the specified path. The device must use `libnnsdk.so` for the matching system and architecture. `AmlCompilerCoreGetVersion()` reports the `libaml_compiler_core.so` actually loaded by the process. Verify `libcompiler_adla.so` against the corresponding vendor release information and online-conversion log. Do not mix Android, Buildroot, and Armbian/Yocto libraries, or mix 32-bit and 64-bit libraries.

2.2 System environment

Identify the target system and architecture so that runtime libraries and a cross-compilation toolchain matching the target system and bit width can be selected in later sections.

Step 1: Determine whether the platform is Android or Linux

Command: `adb shell getprop ro.product.cpu.abi`

Result:

- Output `arm64-v8a`: 64-bit Android
- Output `armeabi-v7a`: 32-bit Android
- Error `command not found`: Linux; continue with Step 2

Step 2: Further identify a Linux system (Buildroot / Yocto / Armbian)

Step 2.1: Check whether the system is Armbian / Yocto

Command: `cat /version.txt`

Example output (Armbian):

```
imagename:a9-by401-bookworm-20260707030949
BRANCH=
VERSION=Linux/Debian/ArmBian-PB#786
SPIN=Linux/Debian/ArmBian-PB#786
BUILD_TIME="2026-07-07 03:27:41"
Generated on Tue Jul  7 03:27:41 UTC 2026

Compiler:
aarch64-none-linux-gnu-gcc (Arm GNU Toolchain 11.3.Rel1) 11.3.1 20220712
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Result:

- **The file exists:** The system is Armbian/Yocto. Use the compiler information to determine whether it is 32-bit or 64-bit. The example above is **Armbian 64-bit (AArch64)**.
- **The file does not exist:** The command fails and the system is considered Buildroot. Continue with Step 2.2.

> **Note:** This identification method applies only to boards using official Amlogic releases. For third-party boards, confirm whether the same version file and identification rules are supported.

Step 2.2: Determine the Buildroot architecture

Command: `adb shell uname -m`

Result:

- `aarch64`: 64-bit Buildroot
- `armv7l`: 32-bit Buildroot

2.3 Platform identification

This ONNX Runtime deployment supports the A311Y3 platform. Confirm that the target platform is A311Y3 before proceeding. Two independent methods are provided below. Compare the result with the **Part_number / Serial mapping table**:

Part_number	Serial
A311Y3	Serial: **4d0a**01020000000070e84a

Method 1: Identify the Part_number

Read the Part_number printed on the board hardware and confirm that it is A311Y3.

Method 2: Identify the Serial number

After the board boots, connect through ADB and run either command:

```
adb shell cat /proc/cpuinfo
# or
adb shell cat /proc/cpu_chipid
```

Matching rules:

- **Preferred:** Compare the first **four characters** of the Serial number with the table.
- **Fallback:** If no match is found, compare the first **three characters**.

2.4 Quick validation

Step 1: Obtain a native case

Download the **64-bit** native case for the target system. The three packages are not interchangeable:

Target system	Download
Android	onnxruntime_android64.tar.gz
Buildroot	onnxruntime_buildroot64.tar.gz
Armbian/Yocto	onnxruntime_armbian_yocto64.tar.gz

Step 2: Run the native case

Download and extract the appropriate resources on the development host. Transfer the complete test directory to the matching target device, then follow the **`Readme.txt` included in the package** to run the test and evaluate the result. The device must provide a compatible `libnnsdk.so`, NPU driver, and runtime permissions. The native case provides a quick check of the device runtime path. For a 32-bit target, build and test the corresponding ABI by following Chapter 3.

3 ONNX Runtime C/C++ deployment

3.1 Install build toolchains

3.1.1 Install CMake and host tools

An Ubuntu 22.04/24.04 `x86_64` development host is recommended. Install the common build tools:

```
sudo apt-get update
sudo apt-get install -y \
  build-essential binutils file git git-lfs curl unzip zip rsync openssh-
client \
  python3 python3-venv python3-pip ninja-build pkg-config
```

ONNX Runtime 1.31.0 requires CMake 3.28 or later during configuration. If the system package is too old, install CMake in an isolated Python virtual environment instead of replacing the system CMake:

```
python3 -m venv "$REPO_ROOT/.venv-build"
source "$REPO_ROOT/.venv-build/bin/activate"
python -m pip install --upgrade pip
python -m pip install 'cmake>=3.28' ninja packaging
```

Android cross-compilation also requires the Android SDK and NDK r27c. See Section 3.1.2.1.

By default, the build scripts use the public headers included with the source. To use other compatible headers, set `AMLNN_NNSDK2_INCLUDE_DIR` and `AMLNN_AML_COMPILER_CORE_INCLUDE_DIR`.

3.1.2 Install cross-compilation toolchains

Perform the following steps on a Linux `x86_64` development host. The toolchain must match the CPU, glibc, libstdc++, and ABI of the target system. The versions below are the validated project baselines. If the target board release specifies a version, use that version instead.

3.1.2.1 Android: install and configure the NDK

Obtain `sdkmanager` from the official Android command-line tools and install version `27.2.12479018` as specified by the official NDK r27c release. If `android-ndk-r27c` is already extracted separately, point `ANDROID_NDK_PATH` to it. Otherwise, run the following from an SDK directory that contains the command-line tools:

```
export ANDROID_SDK_PATH=/path/to/android-sdk
SDKMANAGER="$ANDROID_SDK_PATH/cmdline-tools/latest/bin/sdkmanager"
"$SDKMANAGER" --sdk_root="$ANDROID_SDK_PATH" --install \
```

```
  'ndk;27.2.12479018' 'platform-tools'
export ANDROID_NDK_PATH="$ANDROID_SDK_PATH/ndk/27.2.12479018"
```

Android 32-bit and 64-bit builds use one NDK r27c installation; separate installations are not required. Both ABIs use Android API 21:

Architecture	ABI	NDK C++ compiler entry point
64-bit	`arm64-v8a`	`aarch64-linux-android21-clang++`
32-bit	`armeabi-v7a`	`armv7a-linux-androideabi21-clang++`

The device must support the corresponding ABI and provide a compatible `libnnsdk.so` under `/vendor/lib64` or `/vendor/lib`. Install the Android SDK and NDK on the development host. The ADB server connected to the device may run on another machine.

3.1.2.2 Buildroot: install a GCC cross-toolchain

Prefer the cross-toolchain specified by the target board release. The validated 32-bit and 64-bit baseline is the Linux `x86_64` host edition of Arm GNU Toolchain 13.2.Rel1. **Do not download a `*-none-elf` bare-metal toolchain.**

Architecture	Toolchain target	Variable used by the build
64-bit	`aarch64-none-linux-gnu`	`BUILDROOT64_PATH`
32-bit	`arm-none-linux-gnueabihf`	`BUILDROOT32_PATH`

Download the package for the target architecture and verify the publisher's checksum. First prepare the installation directory:

```
TOOLCHAIN_PARENT=/path/to/toolchains
mkdir -p "$TOOLCHAIN_PARENT"
```

64-bit (aarch64)

```
tar -xf /path/to/arm-gnu-toolchain-13.2.rel1-x86_64-aarch64-none-linux-
gnu.tar.xz -C "$TOOLCHAIN_PARENT"
```

32-bit (ARMv7)

```
tar -xf /path/to/arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-linux-
gnueabihf.tar.xz -C "$TOOLCHAIN_PARENT"
```

Run only the extraction command for the target architecture. This version is the validated project baseline. If the Buildroot target uses a different version of glibc or libstdc++, use a toolchain that matches its release and

confirm that the target provides all symbol versions required by the generated program.

3.1.2.3 Armbian/Yocto: install a Poky SDK

Obtain the Poky SDK that matches the target board from the vendor/BSP release. Do not use the Arm GNU toolchain or Android NDK as a Yocto SDK. This project uses Poky SDK 4.0.20. Reference installers for the corresponding release are listed below; after downloading, confirm that the target board release and SDK version match:

- 64-bit Poky SDK 4.0.20
- 32-bit Poky SDK 4.0.20

Run only the installer for the target architecture. Install the two SDKs into different, new directories:

```
# 64-bit
export YOCTO64_PATH=/path/to/new/yocto64-sdk
sh /path/to/yocto64-sdk-installer.sh -y -d "$YOCTO64_PATH"

# 32-bit
export YOCTO32_PATH=/path/to/new/yocto32-sdk
sh /path/to/yocto32-sdk-installer.sh -y -d "$YOCTO32_PATH"
```

The 32-bit environment file is `environment-setup-armv7at2hf-neon-poky-linux-gnueabi`, and the 64-bit file is `environment-setup-armv8a-poky-linux`. The installer must complete path relocation successfully. If the vendor installer does not support `-y -d`, follow its instructions to specify a separate installation directory. The `yocto/{32,64}/build_so.sh` scripts load the environment file themselves; do not manually `source` it first.

3.2 Build and run ONNX Runtime

3.2.1 Source download

Source code: aml-onnxruntime

Build instructions and demo tools: amlnn-third-party-runtime/onnxruntime

Libraries: amlnn-toolkit/amlnn_runtime/nn_runtime

Obtain the source code, guidance tools, and vendor libraries in the following order, then select the target system and architecture for the build.

Preparation Step 1: Execution conventions

- **Execution location:** Unless a heading explicitly says "on the target device," run commands on a Linux `x86_64` development host. Run device commands on the target device.
- **Launching scripts:** Run all `.sh` build scripts with `bash path/to/script.sh`. **Do not use `source script.sh` or `. script.sh`**, because doing so may affect the current interactive shell.
- **Placeholders:** Replace `/path/to/...` and `board-address` with actual paths or addresses. Do not run them literally. Build and packaging scripts report missing inputs.

- **Build directories:** Use a different `BUILD_DIR` for each platform, ABI, and converter configuration. The default script directories are separated by platform. Never point Android and Linux cross-builds at the same directory.

Preparation Step 2: Obtain the source code

```
git clone -b amlogic/aml-npu-support \
  https://github.com/Amlogic-NN/aml-onnxruntime.git aml-onnxruntime
export REPO_ROOT="$(cd aml-onnxruntime && pwd -P)"
git -C "$REPO_ROOT" checkout --detach
08a78e6581a877aeab5d7036ca6d45c9cfdefc5f
git -C "$REPO_ROOT" submodule update --init --recursive
```

Preparation Step 3: Obtain and install AMLNN Release Tools

Obtain the complete `onnxruntime/` directory from the guidance repository and place it at `tools/amlnn_release/` in the source tree:

```
cd "$(dirname "$REPO_ROOT")"
git clone https://github.com/Amlogic-NN/amlnn-third-party-runtime.git
(
  set -e
  test ! -e "$REPO_ROOT/tools/amlnn_release" && \
    test ! -L "$REPO_ROOT/tools/amlnn_release"
  cp -a amlnn-third-party-runtime/onnxruntime \
    "$REPO_ROOT/tools/amlnn_release"
  bash "$REPO_ROOT/tools/amlnn_release/common/verify_source_tree.sh"
)
```

The resulting layout is:

```
aml-onnxruntime/
└── tools/amlnn_release/
    ├── android/{32,64}/
    ├── linux/{32,64}/
    ├── yocto/{32,64}/
    ├── common/
    └── pipeline_demo/
```

Preparation Step 4: Obtain runtime libraries

The `nn_runtime` directory in amlnn-toolkit provides 32-bit and 64-bit Core, Compiler, and NNSDK libraries for each platform. Core and Compiler are in the same ABI directory and can be used directly by the build scripts without rearranging them.

```
cd "$(dirname "$REPO_ROOT")"
git lfs install
git clone https://github.com/Amlogic-NN/amlnn-toolkit.git
git -C amlnn-toolkit lfs pull --include='amlnn_runtime/nn_runtime/**'
export NN_RUNTIME_ROOT="$(cd amlnn-toolkit/amlnn_runtime/nn_runtime && pwd -
P)"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
```

All paths below are relative to `NN_RUNTIME_ROOT`. In each row, the Core/Compiler directory contains both `libaml_compiler_core.so` and `libcompiler_adla.so`. The NNSDK library is named `libnnsdk.so`.

Target platform	Core / Compiler directory	NNSDK directory
Android 64-bit	`aml_compiler_core/libs/android/arm64-v8a`	`nnsdk/android/lib64`
Android 32-bit	`aml_compiler_core/libs/android/armeabi-v7a`	`nnsdk/android/lib32`
Buildroot 64-bit	`aml_compiler_core/libs/linux/aarch64`	`nnsdk/linux/buildroot/lib64`
Buildroot 32-bit	`aml_compiler_core/libs/linux/armv7`	`nnsdk/linux/buildroot/lib32`
Armbian/Yocto 64-bit	`aml_compiler_core/libs/yocto/aarch64`	`nnsdk/linux/yocto/lib64`
Armbian/Yocto 32-bit	`aml_compiler_core/libs/yocto/armv7`	`nnsdk/linux/yocto/lib32`

If the target already provides a compatible `libnnsdk.so`, use the device version. Otherwise, the device integrator must deploy the version matching the target system and architecture. Do not mix libraries from different platforms or architectures.

For the target architecture, build the shared library, demo, and runtime package in sequence, then run the package on the device. In a new terminal, first set the common path variables below. For Android, also set `ANDROID_SDK_PATH` and `ANDROID_NDK_PATH` from Section 3.1.2.1. Set the Buildroot and Armbian/Yocto toolchain variables in the corresponding platform sections.

```
export REPO_ROOT=/absolute/path/to/aml-onnxruntime
export NN_RUNTIME_ROOT=/absolute/path/to/amlnn-
toolkit/amlnn_runtime/nn_runtime
```

3.2.2 Build and run

3.2.2.1 Build and run on Android

Step 1: Build the shared library (32/64-bit)

64-bit (arm64-v8a)

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-android64"
bash tools/amlnn_release/android/64/build_so.sh
```

32-bit (armeabi-v7a): Use Android NDK r27c. The device must support `armeabi-v7a` and provide `/system/bin/linker` and `/vendor/lib/libnnsdk.so`.

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-android32"
bash tools/amlnn_release/android/32/build_so.sh
```

The output is `$BUILD_DIR/Release/libonnxruntime.so`. Do not share a build directory between 32-bit and 64-bit builds. When host resources permit, set `PARALLEL_JOBS=8`.

Step 2: Build the demo and runtime package (32/64-bit)

The demo must use an ONNX Runtime build for the same platform and architecture. The commands below explicitly set `ORT_BUILD_DIR`.

64-bit (arm64-v8a)

```
cd "$REPO_ROOT"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-android64"
ORT_BUILD_DIR="$BUILD_DIR/Release" bash
tools/amlnn_release/android/64/build_demo.sh
```

The demo output is under `tools/amlnn_release/pipeline_demo/bin/android64/` and includes `pipeline_demo` and `libonnxruntime.so`. Next, create the runtime package. `AMLNN_VENDOR_SDK_ROOT` must point to the runtime library root, and the output package must use a new directory outside the source tree.

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-android-package.XXXXXX)"
DEMO_PACKAGE_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/android64
"
PACKAGE_DIR="$DEMO_PACKAGE_DIR" OUTPUT_DIR="$PACKAGE_PARENT/android64" bash
tools/amlnn_release/android/64/prepare_package.sh
(cd "$PACKAGE_PARENT/android64" && sha256sum -c SHA256SUMS)
```

32-bit (armeabi-v7a): In a new shell, set a separate `BUILD_DIR` and run:

```
cd "$REPO_ROOT"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-android32"
ORT_BUILD_DIR="$BUILD_DIR/Release" bash
tools/amlnn_release/android/32/build_demo.sh
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-android-package.XXXXXX)"
DEMO_PACKAGE_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/android32
"
PACKAGE_DIR="$DEMO_PACKAGE_DIR" OUTPUT_DIR="$PACKAGE_PARENT/android32" bash
tools/amlnn_release/android/32/prepare_package.sh
(cd "$PACKAGE_PARENT/android32" && sha256sum -c SHA256SUMS)
```

The demo is under `tools/amlnn_release/pipeline_demo/bin/android32/`.

Step 3: Run on the device (32/64-bit)

First complete Step 2 for the required architecture in Section 3.2.2.1, then transfer the complete package and ONNX model to the Android device.

Deployment requirements

- The package contains `pipeline_demo`, `libonnxruntime.so`, the Core/Compiler libraries for the same ABI, `libc++_shared.so`, `run_demo.sh`, the batch script, and `SHA256SUMS`.
- The device must provide an NNSDK and NPU driver for the same ABI.
- Do not mix 32-bit and 64-bit packages.

Run the following in a **device terminal**. Select `/vendor/lib64` or `/vendor/lib` for the actual architecture:

```
PACKAGE_DIR=/path/on/device/to/android64-or-android32
NNSDK_LIB_DIR=/vendor/lib64  # Use /vendor/lib for 32-bit Android
MODEL=/path/on/device/to/model.onnx
OUTPUT_DIR=/path/on/device/to/new-output-directory
(cd "$PACKAGE_DIR" && sha256sum -c SHA256SUMS)
NNSDK_LIB_DIR="$NNSDK_LIB_DIR" sh "$PACKAGE_DIR/run_demo.sh" "$MODEL"
"$OUTPUT_DIR" error
```

Result evaluation

- The log level may be `error` (minimum logging; **it does not indicate an error state**), `info`, or `debug`. `warning` is not supported and is explicitly rejected.
- `OUTPUT_DIR` must not already exist, and its parent must be writable. If no input files are supplied, the demo uses zero-filled inputs and validates only the execution path. For real inputs, append raw input files such as `input0.bin` after `error` in model-input order.
- `run_demo.sh` checks for a non-empty generated ADLA file, `output_0.bin`, and `output_0.meta`. Compare output shape, data type, and numerical accuracy with the model reference separately.

Optional ADB shortcut: If the host is connected to an Android device, run `tools/amlnn_release/android/64/run_demo.sh` directly, or use `android/32/run_demo.sh` for 32-bit. The argument order is `model-path output-directory [log-level] [input-files...]`. The script pushes the demo, ONNX Runtime, Core, Compiler, `libc++_shared.so`, and the model to a temporary device directory, then runs the test. NNSDK is loaded from `/vendor/lib64` or `/vendor/lib`. By default, the script uses `libc++_shared.so` under `ANDROID_NDK_PATH`; set `AMLNN_LIBCXX_SO` to override it. For multiple devices, set `ANDROID_SERIAL` first.

3.2.2.2 Build and run on Buildroot

Step 1: Build the shared library (32/64-bit)

`linux/32` and `linux/64` use the paired vendor SDK libraries under `linux/armv7` and `linux/aarch64`, respectively. The cross-toolchain must match the target glibc, libstdc++, and ABI; do not infer compatibility from the directory name alone. The validated 32/64-bit baseline uses Arm GNU Toolchain 13.2.Rel1. `BUILDROOT{32,64}_PATH` points to the **extracted toolchain root**.

64-bit (aarch64)

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILDROOT64_PATH=/path/to/arm-gnu-toolchain-13.2.Rel1-x86_64-aarch64-
none-linux-gnu
export PATH="$BUILDROOT64_PATH/bin:$PATH"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-buildroot64"
export PARALLEL_JOBS=8
bash tools/amlnn_release/linux/64/build_so.sh
```

32-bit (ARMv7): Use a separate shell, toolchain, and build directory. Do not reuse the 64-bit CMake cache.

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILDROOT32_PATH=/path/to/arm-gnu-toolchain-13.2.Rel1-x86_64-arm-
none-linux-gnueabihf
export PATH="$BUILDROOT32_PATH/bin:$PATH"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-buildroot32"
export PARALLEL_JOBS=8
bash tools/amlnn_release/linux/32/build_so.sh
```

The output is `$BUILD_DIR/Release/libonnxruntime.so`. The packaging script checks the target architecture.

Step 2: Build the demo and runtime package (32/64-bit)

64-bit (aarch64)

```
cd "$REPO_ROOT"
export BUILDROOT64_PATH=/path/to/arm-gnu-toolchain-13.2.Rel1-x86_64-aarch64-
none-linux-gnu
export PATH="$BUILDROOT64_PATH/bin:$PATH"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-buildroot64"
DEMO_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/linux64"
ORT_BUILD_DIR="$BUILD_DIR/Release" OUTPUT_DIR="$DEMO_DIR" bash
tools/amlnn_release/linux/64/build_demo.sh
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-linux64-package.XXXXXX)"
ORT_BUILD_DIR="$BUILD_DIR/Release" PIPELINE_DEMO="$DEMO_DIR/pipeline_demo" \
  OUTPUT_DIR="$PACKAGE_PARENT/linux64" bash
tools/amlnn_release/linux/64/prepare_package.sh
echo "PACKAGE_DIR=$PACKAGE_PARENT/linux64"
```

32-bit (ARMv7): In a separate shell, set `BUILDROOT32_PATH` and an independent `BUILD_DIR` as described in Step 1 of Section 3.2.2.2, then run:

```
cd "$REPO_ROOT"
export BUILDROOT32_PATH=/path/to/arm-gnu-toolchain-13.2.Rel1-x86_64-arm-
none-linux-gnueabihf
export PATH="$BUILDROOT32_PATH/bin:$PATH"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-buildroot32"
DEMO_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/linux32"
ORT_BUILD_DIR="$BUILD_DIR/Release" OUTPUT_DIR="$DEMO_DIR" bash
tools/amlnn_release/linux/32/build_demo.sh
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-linux32-package.XXXXXX)"
ORT_BUILD_DIR="$BUILD_DIR/Release" PIPELINE_DEMO="$DEMO_DIR/pipeline_demo" \
  OUTPUT_DIR="$PACKAGE_PARENT/linux32" bash
tools/amlnn_release/linux/32/prepare_package.sh
echo "PACKAGE_DIR=$PACKAGE_PARENT/linux32"
```

Notes

- The packaging scripts use `readelf` to verify the ELF architecture and bitness of ONNX Runtime, Core, Compiler, and the demo, and generate `SHA256SUMS`.
- The runtime package does not include the target-provided `libnnsdk.so`.

Step 3: Run on the device (32/64-bit)

First complete Step 2 for the required architecture in Section 3.2.2.2. The demo and ONNX Runtime library must use the same source, cross-toolchain, and ABI.

Runtime package contents (**excluding** the target `libnnsdk.so`, NPU driver, and model)

- `pipeline_demo`, `run_demo.sh`, and `run_batch.sh`
- `libonnxruntime.so` and `libonnxruntime.so.1`
- `libaml_compiler_core.so` and `libcompiler_adla.so` for the same platform and architecture

- `SHA256SUMS` (the packaging script verifies the ELF architecture and bitness of the three shared libraries and demo)

Deployment requirements

- Transfer the complete package to an executable directory on the matching target board using `tar`/`scp` or another project-specific method. After extraction, run `sha256sum -c SHA256SUMS` in the package directory.
- A 32-bit board requires a 32-bit package, and a 64-bit board requires a 64-bit package.
- The target must provide compatible glibc, libstdc++, NNSDK, NPU driver, and permissions, as well as a readable ONNX model and writable output parent directory.

Run on the target: `NNSDK_LIB_DIR` points to the directory containing the target-provided `libnnsdk.so`, such as `/usr/lib`. Do not copy a library from another platform.

```
PACKAGE_DIR=/path/on/board/to/linux64-or-linux32
NNSDK_LIB_DIR=/usr/lib
MODEL=/path/on/board/to/model.onnx
(cd "$PACKAGE_DIR" && sha256sum -c SHA256SUMS)
OUTPUT_PARENT="$(mktemp -d /tmp/amlnn-smoke.XXXXXX)"
OUTPUT_DIR="$OUTPUT_PARENT/output"
NNSDK_LIB_DIR="$NNSDK_LIB_DIR" sh "$PACKAGE_DIR/run_demo.sh" "$MODEL"
"$OUTPUT_DIR" error
```

Result evaluation

- Compare output shape, data type, and numerical values with the reference separately. A nonzero exit code, missing ADLA file, or missing output file is a failure.
- `run_demo.sh` creates a new output directory, performs online conversion and inference, and verifies a non-empty ADLA file, `output_0.bin`, and `output_0.meta`. It rejects an existing output directory to prevent overwriting previous results.

Batch testing: The script processes only `.onnx` files directly under the model directory and starts a separate process for each model. It does not delete models or intermediate artifacts. If a model fails, the failure is logged and testing continues with the remaining models.

```
MODEL_DIR=/path/on/board/to/onnx-models
BATCH_PARENT="$(mktemp -d /tmp/amlnn-batch.XXXXXX)"
NNSDK_LIB_DIR="$NNSDK_LIB_DIR" sh "$PACKAGE_DIR/run_batch.sh" "$MODEL_DIR"
"$BATCH_PARENT/results"
# Inspect $BATCH_PARENT/results/manifest.tsv and each model's .log file.
```

The exit code is `0` when all models succeed and `1` when any model fails. The result root directory must not already exist.

3.2.2.3 Build and run on Armbian/Yocto

Step 1: Build the shared library (32/64-bit)

The build scripts load the Poky SDK themselves. Do not manually `source` the SDK before running them. Use a separate `BUILD_DIR` for each architecture.

64-bit (aarch64): Use the Poky SDK that matches the target system. See Section 3.1.2.3 for installation.

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export YOCTO64_PATH=/path/to/installed/amlogic-yocto64-sdk
export BUILD_DIR="$REPO_ROOT/build/AMLNN-yocto64"
export PARALLEL_JOBS=8  # Use 1 if memory is insufficient
env -u LD_LIBRARY_PATH bash tools/amlnn_release/yocto/64/build_so.sh
```

The output is under `$BUILD_DIR/Release/`. Do not substitute Linux/Buildroot libraries.

32-bit (ARMv7): Use a vendor Poky/Yocto32 SDK matching the target and install it as described in Section 3.1.2.3. The validated project baseline uses SDK 4.0.20 and `arm-poky-linux-gnueabi-g++` 11.4.0.

```
cd "$REPO_ROOT"
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export YOCTO32_PATH=/path/to/installed/amlogic-yocto32-sdk
export BUILD_DIR="$REPO_ROOT/build/AMLNN-yocto32"
export PARALLEL_JOBS=8  # Use 1 if memory is insufficient
env -u LD_LIBRARY_PATH bash tools/amlnn_release/yocto/32/build_so.sh
```

The output is under `$BUILD_DIR/Release/`.

Step 2: Build the demo and runtime package (32/64-bit)

Select the runtime libraries from `aml_compiler_core/libs/yocto/{aarch64,armv7}`. Core and Compiler are in the same ABI directory. Do not substitute Linux/Buildroot libraries.

64-bit (aarch64): Cross-compile the demo with the same Yocto64 SDK and create a complete runtime package. `BUILD_DIR` must be the directory used to build the shared library in Step 1 of Section 3.2.2.3, and `OUTPUT_DIR` must be a new package directory.

```
cd "$REPO_ROOT"
export YOCTO64_PATH=/path/to/installed/amlogic-yocto64-sdk
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-yocto64"
DEMO_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/yocto64"
env -u LD_LIBRARY_PATH OUTPUT_DIR="$DEMO_DIR" bash
tools/amlnn_release/yocto/64/build_demo_cross.sh
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-yocto64-package.XXXXXX)"
ORT_BUILD_DIR="$BUILD_DIR/Release" PIPELINE_DEMO="$DEMO_DIR/pipeline_demo" \
  OUTPUT_DIR="$PACKAGE_PARENT/yocto64" bash
tools/amlnn_release/yocto/64/prepare_package.sh
echo "PACKAGE_DIR=$PACKAGE_PARENT/yocto64"
```

32-bit (ARMv7): Do not `source` the SDK first.

```
cd "$REPO_ROOT"
export YOCTO32_PATH=/path/to/installed/amlogic-yocto32-sdk
export AMLNN_VENDOR_SDK_ROOT="$NN_RUNTIME_ROOT/aml_compiler_core"
export BUILD_DIR="$REPO_ROOT/build/AMLNN-yocto32"
DEMO_DIR="$REPO_ROOT/tools/amlnn_release/pipeline_demo/bin/yocto32"
env -u LD_LIBRARY_PATH OUTPUT_DIR="$DEMO_DIR" bash
tools/amlnn_release/yocto/32/build_demo.sh
PACKAGE_PARENT="$(mktemp -d /tmp/amlnn-yocto32-package.XXXXXX)"
ORT_BUILD_DIR="$BUILD_DIR/Release" PIPELINE_DEMO="$DEMO_DIR/pipeline_demo" \
  OUTPUT_DIR="$PACKAGE_PARENT/yocto32" bash
tools/amlnn_release/yocto/32/prepare_package.sh
echo "PACKAGE_DIR=$PACKAGE_PARENT/yocto32"
```

Note: Packaging requires `PIPELINE_DEMO`, `ORT_BUILD_DIR`, and a new `OUTPUT_DIR`. Providing only the output directory is not sufficient.

Step 3: Run on the device (32/64-bit)

First complete Step 2 for the required architecture in Section 3.2.2.3. The demo and ONNX Runtime library must use the same source, vendor Poky SDK, and ABI.

Runtime package contents (**excluding** the target `libnnsdk.so`, NPU driver, and model)

- `pipeline_demo`, `run_demo.sh`, and `run_batch.sh`
- `libonnxruntime.so` and `libonnxruntime.so.1`
- `libaml_compiler_core.so` and `libcompiler_adla.so` for the same platform and architecture
- `SHA256SUMS` (the packaging script verifies the ELF architecture and bitness of the three shared libraries and demo)

Deployment requirements

- Transfer the complete package to an executable directory on the matching target board using `tar`/`scp` or another project-specific method. After extraction, run `sha256sum -c SHA256SUMS` in the package directory.
- A 32-bit board requires a 32-bit package, and a 64-bit board requires a 64-bit package.
- The target must provide compatible glibc, libstdc++, NNSDK, NPU driver, and permissions, as well as a readable ONNX model and writable output parent directory.

Run on the target: `NNSDK_LIB_DIR` points to the directory containing the target-provided `libnnsdk.so`, such as `/usr/lib`. Do not copy a library from another platform.

```
PACKAGE_DIR=/path/on/board/to/yocto64-or-yocto32
NNSDK_LIB_DIR=/usr/lib
MODEL=/path/on/board/to/model.onnx
(cd "$PACKAGE_DIR" && sha256sum -c SHA256SUMS)
OUTPUT_PARENT="$(mktemp -d /tmp/amlnn-smoke.XXXXXX)"
OUTPUT_DIR="$OUTPUT_PARENT/output"
```

```
NNSDK_LIB_DIR="$NNSDK_LIB_DIR" sh "$PACKAGE_DIR/run_demo.sh" "$MODEL"
"$OUTPUT_DIR" error
```

Result evaluation

- Compare output shape, data type, and numerical values with the reference separately. A nonzero exit code, missing ADLA file, or missing output file is a failure.
- `run_demo.sh` creates a new output directory, performs online conversion and inference, and verifies a non-empty ADLA file, `output_0.bin`, and `output_0.meta`. It rejects an existing output directory to prevent overwriting previous results.

Batch testing: The script processes only `.onnx` files directly under the model directory and starts a separate process for each model. It does not delete models or intermediate artifacts. If a model fails, the failure is logged and testing continues with the remaining models.

```
MODEL_DIR=/path/on/board/to/onnx-models
BATCH_PARENT="$(mktemp -d /tmp/amlnn-batch.XXXXXX)"
NNSDK_LIB_DIR="$NNSDK_LIB_DIR" sh "$PACKAGE_DIR/run_batch.sh" "$MODEL_DIR"
"$BATCH_PARENT/results"
# Inspect $BATCH_PARENT/results/manifest.tsv and each model's .log file.
```

The exit code is `0` when all models succeed and `1` when any model fails. The result root directory must not already exist.

4 Supported models

The following table lists the ONNX models supported by this release, grouped by major use case.

Use case	Model family	Supported models
Image classification and recognition	AlexNet / CaffeNet	`bvlcalexnet-7.onnx` `caffenet-7.onnx`
	DenseNet / EfficientNet	`densenet-7.onnx` `efficientnet_b0.onnx`
	GoogLeNet / Inception	`googlenet-7.onnx` `inception-v1-7.onnx`
	MobileNet	`mobilenet_v2.onnx` `mobilenetv2-7.onnx`

	ResNet	`resnet18.onnx`, `resnet18-v1-7.onnx`, `resnet18-v2-7.onnx`
		`resnet34-v1-7.onnx`, `resnet34-v2-7.onnx`
		`resnet50-v1-12.onnx`, `resnet50-v1-7.onnx`, `resnet50-v2-7.onnx`
		`resnet101-v1-7.onnx`, `resnet101-v2-7.onnx`
		`resnet152-v1-7.onnx`, `resnet152-v2-7.onnx`

	ShuffleNet / SqueezeNet	`shufflenet-v2-10.onnx`
		`squeezenet1.0-12.onnx`
		`squeezenet1.1-7.onnx`

	VGG	`vgg16-7.onnx`
		`vgg19-7.onnx`
		`vgg19-bn-7.onnx`

	Other	`mnist-7.onnx`, `scene-middle.onnx`
		`age_googlenet.onnx`, `emotion-ferplus-7.onnx`
		`edge256_ped_reid_mbv1_20200815.onnx`

Image segmentation	Human Segmentation	`paddle_human_seg.onnx`
Object detection	YOLOv2	`yolov2-coco-9.onnx`
Basic operator validation	Mul	`hpmul.onnx`

5 FAQ

Symptom	Check and resolution
The build reports that the AMLNN Provider is missing, or source verification fails	Verify the source revision and the location of `tools/amlnn_release/`. The unmodified upstream ONNX Runtime source does not contain this integration.
CMake generator conflict	Use a new `BUILD_DIR` for each generator, system, and architecture. Do not reuse an old cache.
Core or Compiler cannot be found	Verify the target platform and ABI, confirm that the paired libraries come from the same release, and check `SHA256SUMS` in the package.
`libnnsdk.so` cannot be found	Confirm that the target provides a compatible NNSDK and that `NNSDK_LIB_DIR` points to the directory containing the library.
Android cannot find `libc++_shared.so`	Use the library from the same NDK used for the build and confirm that it is included in the complete runtime package.
The Linux loader cannot find `libonnxruntime.so.1`	Package with the platform-specific `prepare_package.sh`. The package must contain the SONAME link.
A model exits with code `137`	Check system logs to determine whether the OOM Killer terminated the process. Preserve the per-model log and reproduce the model separately.
Only `output_0.bin` is present and the output meaning is unclear	Read `output_0.meta` for the output name, data type, and shape. A zero-filled-input result is not a substitute for numerical validation.